

integration_sec06

6. Phase 5: Provider Factory and Runtime Integration

The provider factory and runtime integration layer serves as the definitive bridge between the LLMsVerifier platform's verification pipeline and HelixTranslate's operational translation infrastructure. While the preceding phases establish the data models, scoring infrastructure, and observability hooks, Phase 5 addresses the critical challenge of transforming verified model data into actionable runtime decisions. This chapter examines the complete refactoring of the existing provider factory to incorporate verification-aware model selection, the construction of a multi-strategy selection engine, the design of resilient fallback and degradation chains, event bus integration for real-time model state propagation, full capability-based model matching across the HelixTranslate ecosystem, and the secure management of provider API keys for more than twenty-five distinct LLM providers.

The architectural premise of Phase 5 is that a verified model registry is only valuable if the runtime translation pipeline can efficiently query it, interpret its scores and capabilities, and act upon changes in model quality without manual operator intervention. The integration introduces a three-layer decision hierarchy: first, the `VerifiedLLMClient` abstraction wraps every provider-backed model with its verification metadata; second, the `SelectionEngine` applies domain-specific strategies to choose the optimal model for a given task; and third, the fallback chain builder ensures that translation requests survive individual model failures through a structured degradation path. Each layer is designed to be independently testable, horizontally scalable, and resilient to the inherently variable performance characteristics of third-party LLM APIs.

6.1 Verified Provider Factory Refactoring

The existing HelixTranslate provider factory, located at `internal/providers/factory.go`, instantiates LLM clients based on static configuration entries in `config/providers.yaml`. This approach treats every configured model as equally suitable for every translation task, with selection limited to round-robin or first-available logic. Phase 5 replaces this flat model with a hierarchical factory that consumes the `VerifiedModelRegistry` populated by the LLMsVerifier ingestion pipeline (Chapter 3) and produces `VerifiedLLMClient` instances enriched with composite scores, tier assignments, capability bitmaps, and verification timestamps.

6.1.1 The VerifiedLLMClient Type

The central abstraction introduced by the refactoring is the `VerifiedLLMClient` struct, which embeds the existing `llm.LLMClient` interface and augments it with verification metadata sourced from the model registry. This design preserves backward compatibility with all existing translation call sites while enabling verification-aware routing decisions at the factory boundary.

```
type VerifiedLLMClient struct {
    llm.LLMClient
    ModelID          string          `json:"model_id"`
    ProviderID       string          `json:"provider_id"`
    VerificationStatus VerificationStatus `json:"verification_status"`
    Score            *CompositeScore `json:"score"`
    Tier             ModelTier       `json:"tier"`
    Capabilities     CapabilityBitmap `json:"capabilities"`
    VerifiedAt       time.Time       `json:"verified_at"`
}

type VerificationStatus string
const (
    StatusPending VerificationStatus = "pending"
    StatusVerified VerificationStatus = "verified"
    StatusFailed   VerificationStatus = "failed"
    StatusRetired  VerificationStatus = "retired"
)

type ModelTier string
const (
    TierPremium ModelTier = "premium"
    TierStandard ModelTier = "standard"
    TierBudget   ModelTier = "budget"
    TierUnranked ModelTier = "unranked"
)
```

The `VerifiedLLMClient` struct is defined in `internal/providers/verified_client.go`. The `llm.LLMClient` embedding ensures that the `VerifiedLLMClient` satisfies the same interface contract as all existing provider implementations, meaning that no changes are required in the core translation engine at `internal/translate/engine.go` or in any of the twenty-seven language-pair adapters. The `VerificationStatus` field captures the lifecycle state of the model as determined by the verifier's continuous evaluation loop: models entering the system begin in `StatusPending`, transition to `StatusVerified` upon passing the minimum score threshold (configurable via `verifier.min_score_threshold`, default `0.65`), move to `StatusFailed` if verification fails after the maximum retry count (default three attempts), and are marked `StatusRetired` if their score drops below the threshold after a prior successful verification. The `ModelTier` field classifies models into four economic tiers. `TierPremium` includes models scoring above `0.90` on the composite index, `TierStandard` covers scores `0.75` to `0.90`, `TierBudget` includes scores `0.65` to `0.75`, and `TierUnranked` captures all other models including those pending verification or those explicitly excluded from tiering. This tier system directly supports the degradation chain described in Section 6.3.

6.1.2 LLMsVerifier Provider Adapter

The factory must support LLMsVerifier as a first-class provider in addition to the existing direct-provider adapters (OpenAI, Anthropic, Google, Azure, Cohere, Mistral, and others). The LLMVerifierAdapter translates between HelixTranslate's `llm.TranslationRequest` type and the LLMsVerifier platform's OpenAI-compatible chat completions endpoint. The adapter resides at `internal/providers/llmsverifier/adapter.go` and implements the full `llm.LLMClient` interface.

```
// internal/providers/llmsverifier/adapter.go
type LLMVerifierAdapter struct {
    baseURL string
    apiKey  string
    modelID string
    client  *http.Client
}

func (a *LLMVerifierAdapter) Translate(ctx context.Context, req
    llm.TranslationRequest) (llm.TranslationResponse, error) {
    payload, _ := json.Marshal(CompletionRequest{
        Model:    a.modelID,
        Messages: convertMessages(req),
    })
    httpReq, _ := http.NewRequestWithContext(ctx, "POST",
        a.baseURL+"/v1/chat/completions",
        bytes.NewReader(payload))
    httpReq.Header.Set("Authorization", "Bearer "+a.apiKey)
    httpReq.Header.Set("Content-Type", "application/json")

    resp, err := a.client.Do(httpReq)
    if err != nil {
        return llm.TranslationResponse{},
            fmt.Errorf("llmsverifier request failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        body, _ := io.ReadAll(resp.Body)
        return llm.TranslationResponse{},
            fmt.Errorf("llmsverifier returned %d: %s",
                resp.StatusCode, body)
    }

    var result CompletionResponse
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil {
        return llm.TranslationResponse{}, fmt.Errorf("decode
            response: %w", err)
    }
    if len(result.Choices) == 0 {
        return llm.TranslationResponse{}, errors.New("empty
            choices in response")
    }
}
```

```

    return llm.TranslationResponse{
        Text:      result.Choices[0].Message.Content,
        TokensUsed: result.Usage.TotalTokens,
    }, nil
}

```

The `convertMessages` helper transforms HelixTranslate's internal message format into the LLMsVerifier chat completions schema, preserving system prompts, user instructions, and any in-context translation examples. The adapter also supports streaming responses via a separate `TranslateStream` method for real-time chat translation tasks, with the stream parser extracting delta chunks from the Server-Sent Events (SSE) response format. The `client` field uses a shared `http.Client` configured with a connection pool of up to one hundred connections and a default request timeout of sixty seconds (configurable via `verifier.request_timeout`). Retry logic with exponential backoff (initial delay one hundred milliseconds, maximum delay five seconds, jitter factor 0.3) is applied at the adapter level for transient errors (HTTP 429, 502, 503, 504).

6.1.3 Factory Integration Points

The refactored factory at `internal/providers/factory.go` introduces three new factory methods: `CreateVerifiedClient` accepts a model identifier and returns a `VerifiedLLMClient` populated from the registry; `CreateBestForTask` accepts a `TaskRequirements` descriptor and delegates to the selection engine (Section 6.2); and `CreateForTier` returns any available client within a specified tier, used primarily by the degradation chain. The factory maintains a hot cache of `VerifiedLLMClient` instances keyed by `model_id`, with entries refreshed every thirty seconds via a background goroutine that polls the `VerifiedModelRegistry` for status changes. Cache invalidation is also triggered by `ModelScoreChangedEvent` notifications on the event bus (Section 6.4), ensuring that score drops or model retirements are reflected in the factory's client pool within sub-second latency.

6.2 Runtime Model Selection Engine

The selection engine is the decision-making core of the runtime integration. Located at `internal/providers/selection/engine.go`, it maps incoming translation tasks to optimal model selections through a pluggable strategy pattern. Each strategy evaluates the set of available verified models against task-specific criteria and returns a ranked or single selection.

6.2.1 Strategy Architecture

The selection engine defines a `SelectionStrategy` interface with a single method `Select(ctx context.Context, candidates []VerifiedLLMClient, task TaskRequirements) (*VerifiedLLMClient, error)`. Six concrete strategy implementations are provided, each targeting a distinct optimization objective. The engine routes tasks to strategies based on the `Priority` field of the `TaskRequirements` descriptor.

Table 1: Task Type to Selection Strategy Mapping

Task Type	Priority	Strategy	Key Criteria
Legal translation	quality	BestScoreStrategy	Capability bitmap includes CapLegalDomain; highest CompositeScore.Overall
Batch processing	cost	LowestCostStrategy	Lowest CompositeScore.CostScore within configured budget ceiling
Real-time chat	speed	FastestStrategy	Lowest measured latency (p95) from CompositeScore.LatencyMs
General text	balanced	BalancedStrategy	Weighted composite of quality, cost, and latency scores
Medical translation	quality	BestScoreStrategy	Capability bitmap includes CapMedicalDomain; highest overall score
Technical documentation	quality	BestScoreStrategy	Capability bitmap includes CapCodeGeneration and CapTechnicalWriting

Additional task types not listed in Table 1 (e.g., literary translation, marketing localization, subtitle timing) follow the same routing logic, with domain-specific capability flags defined in the CapabilityBitmap enumeration. The TaskRequirements struct captures source and target languages, content domain, minimum quality threshold, maximum acceptable latency, and budget constraints, enabling the strategies to make multi-dimensional tradeoffs.

6.2.2 Selection Engine Implementation

The SelectionEngine maintains a registry of named strategies and a reference to the ModelRegistry for querying available models. Its SelectModel method implements the complete selection pipeline: filtering by minimum score, selecting the appropriate strategy, and returning the chosen model.

```

type SelectionEngine struct {
    strategies map[string]SelectionStrategy
    registry   *ModelRegistry
}

func (e *SelectionEngine) SelectModel(ctx context.Context, task
    TaskRequirements) (*VerifiedLLMClient, error) {
    available, err := e.registry.FilterByScore(ctx,
        task.MinQuality)
    if err != nil {
        return nil, fmt.Errorf("filter models: %w", err)
    }
    if len(available) == 0 {
        return nil, ErrNoModelsAvailable
    }
}

```

```

    strategy := e.strategies[task.Priority]
    if strategy == nil {
        strategy = e.strategies["balanced"]
    }
    return strategy.Select(ctx, available, task)
}

```

The `FilterByScore` method on `ModelRegistry` queries the underlying PostgreSQL-backed registry (Chapter 3) with the clause `WHERE overall_score >= $1 AND verification_status = 'verified'`, ensuring that only currently verified models meeting the task's minimum quality bar are considered. The query is indexed on a composite of (`verification_status`, `overall_score DESC`) for sub-millisecond lookup latency on the production dataset of approximately four hundred active models. If `task.Priority` does not match a registered strategy key, the engine defaults to `BalancedStrategy`, which computes a weighted sum of normalized quality, cost, and latency scores using weights configurable via `selection.balanced_weights` (default quality: `0.5`, cost: `0.3`, latency: `0.2`). The `BestScoreStrategy` selects the highest-scoring model that satisfies all required capabilities; in case of a tie, it prefers the model with the lower cost score. The `LowestCostStrategy` selects the cheapest verified model and falls back to the next tier up if no model meets the budget ceiling. The `FastestStrategy` selects the model with the lowest p95 latency measurement from the verifier's benchmarking history, filtering out any model with fewer than ten latency samples to ensure statistical significance.

6.2.3 Strategy Registration and Hot Swapping

Strategies are registered at engine initialization via a builder pattern in `internal/providers/selection/builder.go`. The factory supports hot-swapping of strategy weights at runtime through a configuration watcher on `config/selection.yaml`; changes to strategy parameters are applied without requiring a service restart. The engine also exposes a `SelectWithOverride` method that accepts an explicit strategy name, used by the HelixTranslate admin API for A/B testing and manual model selection experiments.

6.3 Fallback and Degradation Chain

Individual LLM API requests can fail for numerous reasons: rate limits, transient provider outages, content policy rejections, context window overflows, or unexpected response formats. The fallback and degradation chain provides a structured, deterministic path for recovering from these failures while preserving translation quality to the maximum extent possible.

6.3.1 Fallback Chain Construction

The `BuildFallbackChain` function constructs an ordered list of fallback candidates from the set of available verified models. The chain is built once per selection decision and cached for the duration of the translation request.

```

func BuildFallbackChain(models []VerifiedLLMClient, maxLength
    int) []FallbackEntry {
    sort.Slice(models, func(i, j int) bool {
        return models[i].Score.Overall > models[j].Score.Overall
    })

    var chain []FallbackEntry
    lastProvider := ""
    for _, m := range models {
        if len(chain) >= maxLength {
            break
        }
        if m.ProviderID == lastProvider && len(chain) > 0 {
            continue // enforce provider diversity
        }
        chain = append(chain, FallbackEntry{
            ModelID:    m.ModelID,
            ProviderID: m.ProviderID,
            Tier:        m.Tier,
            Score:       m.Score.Overall,
        })
        lastProvider = m.ProviderID
    }
    return chain
}

```

The function sorts all verified models by descending overall score and then iterates to build a chain of up to maxLength entries (default five, configurable via fallback.max_chain_length). The provider diversity constraint ensures that consecutive entries in the chain originate from different infrastructure providers, mitigating the risk of a single provider outage cascading through all fallback attempts. The FallbackEntry struct contains only the fields necessary for retry dispatch: ModelID, ProviderID, Tier, and Score. The full VerifiedLLMClient is retrieved from the factory cache when a fallback entry is activated.

6.3.2 Degradation Policy

The degradation policy defines five distinct levels that a translation request progresses through on repeated failure. Each level specifies a different recovery action, and the policy is applied by the DegradationController at internal/providers/fallback/degradation.go.

```

type DegradationLevel int
const (
    LevelPrimary    DegradationLevel = iota // Best match from
        selection engine
    LevelRetry      // Retry same model
        once (handles transient errors)
    LevelFallback   // Next entry in
        fallback chain
    LevelTierDown   // Drop to next tier
        (e.g., Premium -> Standard)
)

```

```
LevelHumanReview
    translator review
)
```

```
// Queue for human
```

At `LevelPrimary`, the request is sent to the model selected by the selection engine. If this fails with a retryable error (HTTP 429, 502, 503, 504, timeout, or connection reset), the controller advances to `LevelRetry` and reissues the identical request to the same model endpoint. The retry includes exponential backoff with jitter. If the retry also fails, the controller advances to `LevelFallback` and dispatches the request to the next entry in the pre-built fallback chain. If all fallback chain entries are exhausted, the controller advances to `LevelTierDown`, which queries the factory for any available model in the next lower tier. For example, if the original selection was `TierPremium`, `LevelTierDown` attempts `TierStandard`, then `TierBudget`, then `TierUnranked`. At each tier, the best-scoring model in that tier is selected using the same strategy as the original request. If no model in any tier succeeds, the request reaches `LevelHumanReview`, where it is enqueued in the human review pipeline at `internal/humanreview/queue.go`. The queue stores the original request payload, the complete error history from all degradation levels, and a priority score based on the requesting user's service tier.

6.3.3 Degradation Metrics and Circuit Breakers

The degradation controller emits Prometheus metrics at every level transition: `helix_degradation_level_total{level="primary|retry|fallback|tier_down|human_review"}`, `helix_fallback_latency_seconds`, and `helix_human_review_queue_size`. Additionally, a per-provider circuit breaker (implemented using the `sony/gobreaker` library) tracks failure rates over a thirty-second window. If a provider's failure rate exceeds fifty percent, the circuit opens and all models from that provider are temporarily excluded from the fallback chain. The circuit closes after a ten-second half-open period during which a single probe request is permitted. This prevents the degradation chain from repeatedly attempting a clearly degraded provider.

6.4 Event Bus Integration

The event bus integration connects the `LLMsVerifier` platform's score change notifications to `HelixTranslate`'s runtime systems, enabling reactive model management. Two primary event types are handled: `ModelScoreChangedEvent`, which triggers model retirement or re-tiering, and `ModelSelectedEvent`, which is published by the selection engine for observability and billing purposes.

6.4.1 Score Change Event Handler

The `handleModelScoreChanged` method on the `VerifierIntegration` type processes score change events from the `LLMsVerifier` platform. When a model's composite score drops below the minimum threshold, the handler automatically retires the model from the active registry.


```

func (v *VerifierIntegration) handleModelScoreChanged(ctx
    context.Context, evt events.ModelScoreChangedEvent) error
{
    v.logger.Printf("Score change: model=%s old=%.2f new=%.2f",
        evt.ModelID, evt.OldScore, evt.NewScore)

    if evt.NewScore < v.config.MinScoreThreshold {
        v.logger.Printf("Model %s score dropped to %.2f,
            retiring", evt.ModelID, evt.NewScore)
        if err := v.registry.UpdateModelStatus(ctx, evt.ModelID,
            "retired"); err != nil {
            return fmt.Errorf("retire model %s: %w", evt.ModelID,
                err)
        }
        // Invalidate factory cache for this model
        v.factory.InvalidateCache(evt.ModelID)
        // Notify operators via PagerDuty if the model was
        TierPremium
        if evt.OldTier == TierPremium {
            v.alerting.Send(ctx, alerting.Alert{
                Severity: alerting.SeverityWarning,
                Summary:   fmt.Sprintf("Premium model %s retired
                    (score %.2f)", evt.ModelID, evt.NewScore),
            })
        }
        return nil
    }

    // Re-tier the model if score moved across tier boundaries
    newTier := scoreToTier(evt.NewScore)
    if newTier != evt.OldTier {
        if err := v.registry.UpdateModelTier(ctx, evt.ModelID,
            newTier); err != nil {
            return fmt.Errorf("update tier for %s: %w",
                evt.ModelID, err)
        }
        v.factory.InvalidateCache(evt.ModelID)
    }
    return nil
}

```

The handler also detects tier boundary crossings: if a model's score improvement promotes it from TierStandard to TierPremium, the registry and factory cache are updated immediately, making the model available for high-priority tasks. Conversely, a demotion triggers cache invalidation and removes the model from premium-eligible selection pools. Score increases that do not cross tier boundaries are logged at debug level but require no action. The `events.ModelScoreChangedEvent` payload includes `ModelID`, `OldScore`, `NewScore`, `OldTier`, `NewTier`, and `Timestamp`, all sourced from the LLMsVerifier scoring pipeline's output (Chapter 4).

6.4.2 Model Selection Event Publisher

When the selection engine chooses a model for a translation task, it publishes a `ModelSelectedEvent` to the event bus. This event is consumed by the billing service, the analytics pipeline, and the real-time dashboard.

```
func (v *VerifierIntegration) PublishModelSelected(ctx
    context.Context, modelID string, task TaskRequirements) {
    events.Publish(ctx, events.ModelSelectedEvent{
        ModelID:      modelID,
        TaskType:     task.Domain,
        SourceLang:   task.SourceLang,
        TargetLang:   task.TargetLang,
        Timestamp:    time.Now(),
    })
}
```

The `events.Publish` function is backed by Redis Streams (`XADD` to `helix:events:model_selected`) with at-least-once delivery semantics. Consumers include: the billing aggregator, which credits the selected model's provider for the request; the analytics pipeline, which feeds into the BigQuery warehouse for usage pattern analysis; and the WebSocket-based real-time dashboard, which displays the currently active model per language pair. The event schema is versioned (current version 1.2) and includes an `EventID` UUID for idempotency.

6.4.3 Event Subscription Topology

The verifier integration subscribes to three Redis Stream channels: `llmverifier:scores` for score changes, `llmverifier:capabilities` for capability bitmap updates (triggered when new MCP integrations are tested), and `llmverifier:retirements` for explicit retirement commands from the verifier platform's administrative interface. Subscription handlers run in a dedicated goroutine pool of eight workers, with each handler wrapped in a panic recovery wrapper and a timeout context of thirty seconds. Failed event processing results in the event being re-queued with an exponential backoff, and events failing after five retries are written to a dead-letter stream for manual inspection.

6.5 Full Capability Integration

HelixTranslate's translation tasks frequently require models with specific capabilities beyond general text generation. Legal translation demands models with structured output and legal domain knowledge; technical documentation requires code-aware models; and RAG-enhanced translation needs models with large context windows. The capability integration layer ensures that the selection engine can filter models by these specialized requirements.

6.5.1 MCP Integration Catalog

The Model Context Protocol (MCP) integrations extend model capabilities by providing structured tool access. The LLMsVerifier platform tests each model's ability to invoke MCP tools correctly, and the results are encoded in the model's CapabilityBitmap. HelixTranslate leverages these verified capabilities to match models to tasks requiring external tool integration.

Table 2: Key MCP Integrations and Required Capabilities

MCP Name	Purpose	Required Capability	File Path
GitHub	Code repository access and file retrieval	CapFunctionCalling	internal/mcp/github/
Playwright	Browser automation for web content extraction	CapFunctionCalling	internal/mcp/playwright/
Qdrant	Vector search for semantic document retrieval	CapLargeContext	internal/mcp/qdrant/
Redis	Distributed cache and RAG backend storage	CapLargeContext	internal/mcp/redis/
Elasticsearch	Full-text search and RAG document indexing	CapLargeContext	internal/mcp/elastic/
Cloudflare	AI Gateway request routing and caching	CapStreaming	internal/mcp/cloudflare/
AWS	Cloud service integration (S3, Lambda, Bedrock)	CapFunctionCalling	internal/mcp/aws/
Kubernetes	Container management and log extraction	CapFunctionCalling	internal/mcp/k8s/

The full MCP catalog contains thirty-five integrations. The remaining twenty-seven include: internal/mcp/jira/ (issue tracking), internal/mcp/slack/ (messaging), internal/mcp/notion/ (documentation), internal/mcp/figma/ (design asset access), internal/mcp/trello/ (project management), internal/mcp/asana/ (task management), internal/mcp/confluence/ (wiki content), internal/mcp/dropbox/ (file storage), internal/mcp/google_drive/ (cloud storage), internal/mcp/onedrive/ (Microsoft storage), internal/mcp/sharepoint/ (enterprise content), internal/mcp/zendesk/ (support tickets), internal/mcp/intercom/ (customer messaging), internal/mcp/hubspot/ (CRM data), internal/mcp/salesforce/ (CRM integration), internal/mcp/stripe/ (payment data), internal/mcp/twilio/ (communications), internal/mcp/sendgrid/ (email delivery), internal/mcp/shopify/ (e-commerce data), internal/mcp/woocommerce/ (e-commerce), internal/mcp/bigcommerce/ (e-commerce), internal/mcp/square/ (point-of-sale), internal/mcp/paypal/ (payment processing), internal/mcp/mongodb/ (document database), internal/mcp/postgresql/ (relational database), internal/mcp/mysql/ (relational

database), and `internal/mcp/sqlite/` (embedded database). Each MCP integration defines its own `CapabilityRequirements` and validates that the selected model's bitmap includes the necessary flags.

6.5.2 Complete Capability Taxonomy

The full capability taxonomy encompasses seven categories covering all verified model abilities relevant to HelixTranslate's operations.

Table 3: Complete Capability Taxonomy

Category	Count	Key Examples
MCP (Model Context Protocol)	35	GitHub, Playwright, Qdrant, Redis, ES, Cloudflare, AWS, K8s, Jira, Slack, Notion
LSP (Language Server Protocol)	10	Go, Python, TypeScript, Rust, C++, Java, Ruby, PHP, Swift, Kotlin
ACP (Agent Communication Protocol)	1	Inter-agent message passing and delegation
Embeddings	13	OpenAI (<code>text-embedding-3-*</code>), Cohere (<code>embed-*</code>), Jina (<code>jina-embeddings-*</code>), Mistral, BGE, E5, GTE, Voyage, Nomic, Google (<code>textembedding-gecko-*</code>), Amazon Titan, IBM Granite, Snowflake Arctic
RAG (Retrieval-Augmented Generation)	3	Qdrant vector search, Redis semantic cache, Elasticsearch hybrid search
Skills	15+	Translation (all 27 language pairs), Summarization, Code Review, Technical Writing, Legal Analysis, Medical Terminology, Literary Style Transfer, Marketing Localization, Subtitle Timing, Voice-to-Text Alignment, Glossary Enforcement, Style Guide Compliance, Back-translation Verification, Post-editing Support, Quality Estimation
Plugins	8+	Markdown formatter, HTML/XML entity normalizer, Unicode bidirectional text handler, ICU message format converter, XLIFF segment processor, TM (translation memory) matcher, Terminology database connector, Custom preprocessing pipeline

The LSP category enables language-aware translation of code comments, docstrings, and inline documentation. When a task is tagged with domain code, the selection engine requires `CapCodeGeneration` (which implies all LSP capabilities) and routes the request to models verified for the specific programming language of the input. The embeddings category supports the RAG pipeline by ensuring that the embedding model used for document indexing is compatible with the generation model's context window and token format. The skills category represents the highest-level capability mapping, directly corresponding to HelixTranslate's service offerings; each skill is verified by the `LLMsVerifier` platform through targeted benchmark datasets.

6.5.3 Capability Matching Engine

The capability matching engine provides a unified interface for checking whether a model satisfies the requirements of a given task.

```
type CapabilityRequirements struct {
    Required      CapabilityBitmap
    Preferred      CapabilityBitmap
    MinContextWindow int
}

func MatchCapabilities(req CapabilityRequirements, modelCaps
    CapabilityBitmap, contextWindow int) MatchResult {
    if contextWindow < req.MinContextWindow {
        return MatchResult{
            Satisfied: false,
            Reason:    fmt.Sprintf("context window %d < required
%d", contextWindow, req.MinContextWindow),
        }
    }
    missing := req.Required &^ modelCaps
    if missing != 0 {
        return MatchResult{
            Satisfied: false,
            Reason:    fmt.Sprintf("missing required capabilities: %b", missing),
        }
    }
    preferred := req.Preferred & modelCaps
    return MatchResult{
        Satisfied: true,
        Preferred:  preferred,
        PreferredCount: bits.OnesCount64(preferred),
    }
}
```

The `CapabilityBitmap` type is a `uint64` bitfield with fifty-six defined capability flags, leaving eight bits reserved for future expansion. The `MatchCapabilities` function first validates the context window requirement (used to exclude models with insufficient token capacity for long-document translation or RAG contexts), then checks that all required capability bits are present in the model's bitmap. Models that fail the required capability check are filtered out before strategy evaluation. Preferred capabilities contribute to the strategy's scoring function: a model that satisfies more preferred capabilities receives a bonus in the `BalancedStrategy` weight calculation, and `BestScoreStrategy` uses preferred capability count as a tiebreaker when overall scores are within 0.01 of each other.

The `CapabilityBitmap` is defined in `internal/providers/capabilities/bitmap.go` with individual constants such as `CapFunctionCalling = 1 << 0`, `CapLargeContext = 1 << 1`, `CapStreaming = 1 << 2`, `CapCodeGeneration = 1 << 3`, `CapLegalDomain = 1 << 4`, `CapMedicalDomain = 1 << 5`, `CapTechnicalWriting = 1 << 6`, and so on

through the fifty-six defined flags. Capability bitmaps are stored in the `verified_models` table as `BIGINT` columns and are updated by the `LLMsVerifier` ingestion pipeline whenever a new capability test is completed.

6.6 API Key Management

The integration with `LLMsVerifier` and the expanded provider ecosystem requires secure management of more than twenty-five distinct API keys. The key management system, implemented in `internal/providers/keys/manager.go`, provides validation, encrypted storage, rotation, and per-key rate limit tracking.

6.6.1 Key Validation

Every API key is validated at configuration load time using a two-step process: first, a provider-specific regex validates the key format (e.g., OpenAI keys match `^sk-[a-zA-Z0-9]{48}$`, Anthropic keys match `^sk-ant-api03-[a-zA-Z0-9_-]{64,}$`); second, a minimal API call to a low-cost endpoint (typically a models listing or a short completion with `max_tokens: 1`) confirms that the key is active and has remaining quota. Keys that fail either step are flagged with `status: invalid` and excluded from the provider factory's client pool.

The validation functions are organized by provider in `internal/providers/keys/validators/`. Each validator implements the `KeyValidator` interface: `ValidateFormat(key string) error` and `ValidateActive(ctx context.Context, key string) error`. The active validation calls use a shared circuit breaker per provider to prevent validation storms during provider outages.

6.6.2 Secure Storage

All API keys are encrypted at rest using AES-256-GCM with a key encryption key (KEK) stored in HashiCorp Vault. The data encryption key (DEK) is unique per provider and is rotated every thirty days via an automated cron job at `cmd/key-rotation/main.go`. The encrypted key material is stored in the PostgreSQL `provider_api_keys` table with the following schema: `id SERIAL PRIMARY KEY`, `provider_id VARCHAR(64) NOT NULL`, `key_ciphertext BYTEA NOT NULL`, `key_nonce BYTEA NOT NULL`, `status VARCHAR(16) DEFAULT 'active'`, `rate_limit_rpm INTEGER`, `rate_limit_tpm INTEGER`, `created_at TIMESTAMPTZ`, `last_rotated_at TIMESTAMPTZ`, and `expires_at TIMESTAMPTZ`. At runtime, keys are decrypted on-demand and cached in process memory with a ten-minute TTL; the in-memory cache stores only the decrypted key string, protected by Go's garbage collection semantics and process-level isolation.

The twenty-five supported providers and their validation endpoints are documented in `config/providers.yaml`. Each entry specifies the provider ID, base URL, validation endpoint path, validation method, expected success status code, and rate limits. The providers include: OpenAI (`api.openai.com`), Anthropic (`api.anthropic.com`), Google Vertex AI (`us-central1-aiplatform.googleapis.com`), Azure OpenAI (`{resource}.openai.azure.com`), Cohere (`api.cohere.com`), Mistral AI (`api.mistral.ai`), `LLMsVerifier` (`api.llmsverifier.io`), Together AI (`api.together.xyz`), Groq (`api.groq.com`), Perplexity (`api.perplexity.ai`), AI21 Labs (`api.ai21.com`), Fireworks AI (`api.fireworks.ai`), Anyscale

(`api.endpoints.anyscale.com`), DeepInfra (`api.deepinfra.com`), Baseten (`model-{model_id}.api.baseten.co`), Replicate (`api.replicate.com`), Hugging Face (`api-inference.huggingface.co`), OctoAI (`text.octoai.run`), Lepton AI (`{model}.lepton.run`), Cloudflare Workers AI (`api.cloudflare.com`), AWS Bedrock (`bedrock-runtime.{region}.amazonaws.com`), Google Gemini (`generativelanguage.googleapis.com`), Moonshot AI (`api.moonshot.cn`), Zhipu AI (`open.bigmodel.cn`), and 01.AI (`api.01.ai`).

6.6.3 Per-Key Rate Limiting

The key manager maintains a distributed rate limiter per API key using Redis as the coordination backend. The limiter tracks both requests-per-minute (RPM) and tokens-per-minute (TPM) quotas as reported by each provider's response headers (`x-ratelimit-limit-requests`, `x-ratelimit-limit-tokens`, `x-ratelimit-remaining-requests`, `x-ratelimit-remaining-tokens`). When a key's rate limit is exhausted, the manager marks it as `status: rate_limited` for the duration specified in the `Retry-After` header (or a default sixty seconds if absent), and the provider factory routes requests to the next available key for that provider. This multi-key support is essential for high-volume translation pipelines that can exceed the quota of a single API key.

6.6.4 Key Rotation and Audit

Key rotation is triggered automatically when an expiry date approaches (within seven days) or manually via the admin API at `POST /admin/keys/{provider_id}/rotate`. The rotation process generates a new key through the provider's API console (for providers supporting programmatic key generation) or accepts a new key via the admin API, validates it, encrypts it, and atomically swaps the active key reference. Old keys are retained in `status: rotated` for twenty-four hours to allow in-flight requests to complete, then are permanently deleted. All key operations (validation, rotation, status changes, access events) are logged to an append-only audit log stored in S3 with object lock enabled, retaining records for a minimum of one year for compliance purposes.

Phase 5 completes the integration by binding the verified model registry to every operational decision in HelixTranslate's translation pipeline. The refactored provider factory produces `VerifiedLLMClient` instances with full metadata, the selection engine routes tasks through domain-appropriate strategies, the degradation chain ensures resilience against individual model failures, the event bus propagates score changes in real time, the capability matcher enables sophisticated task-to-model assignment across seventy-plus verified abilities, and the key management system securely orchestrates access to more than twenty-five providers. Together, these components transform the LLMsVerifier platform from a passive scoring system into an active participant in HelixTranslate's runtime quality assurance.